

SEEKER: Robot Exploration via Deep Reinforcement Learning with Structure-Aware and Knowledge-Evolving Spatiotemporal Fusion

Abstract—Autonomous exploration of unknown environments remains challenging for mobile robots, primarily due to myopic decision-making and limited long-term planning capability. Existing methods often underutilize structural information and exploration history, leading to redundant revisits and degraded exploration efficiency. To address these issues, we propose SEEKER, a deep reinforcement learning framework that fuses structure-aware spatial features and exploration-knowledge evolution for sustained exploration. Specifically, a structural descriptor is designed based on an encroaching strategy to characterize regional structural properties and encoded by a GNN into spatial features. An incremental Koopman operator-based temporal embedding block models the exploration knowledge evolution through Multi-EMA refinement and inheritance mechanism. The spatial and temporal representations are fused through an attention-integrated policy for non-myopic decision-making, followed by a dual-stage planning scheme that generates local exploration and global guidance paths. Extensive simulations are conducted in three unseen environments. The results demonstrate that SEEKER achieves superior exploration efficiency, consistently high coverage, and runtime balance over three benchmarks. Real-world experiments further validate the practical feasibility of the proposed framework. Our project page is available at: <https://seeker-tii.github.io/SEEKER/>.

Index Terms—Autonomous exploration, mobile robots, reinforcement learning, spatiotemporal fusion.

I. INTRODUCTION

AUTONOMOUS exploration of unknown environments by mobile robots has become one of the most active research topics in recent years, driven by synergistic advances in machine learning, onboard sensing techniques, and autonomous control and planning algorithms [1]. This capability is fundamental to a wide range of industrial intelligent systems, e.g., operations & maintenance, industrial inspection and digital twin [2]–[4], where robots must operate in GPS-denied, varying environments with stringent requirements on exploration efficiency and completeness. Despite the substantial body of research devoted to enhancing exploration performance through modular architecture designs and algorithmic improvements, the majority of existing approaches remain predominantly rule-based. Such methods often suffer from limited global optimality [5], [6], scalability bottlenecks in viewpoint sampling [7], [8], or limited behavioral robustness in diverse and evolving environments [9].

Recently, learning-based methods, particularly deep reinforcement learning (DRL), have emerged as a dominant paradigm for autonomous exploration, enabling agents to learn navigation policies through environment interaction under partial observability. Nevertheless, several fundamental

challenges remain unresolved. For instance, many DRL approaches overlook region-level structural properties or merely preserve the dominant topology [9], [10]. By failing to quantify structural information as key features for computation, these methods often treat viewpoints with identical information gain but varying degrees of unexplored coverage as equivalent, leading to suboptimal decisions and reduced exploration efficiency [11]. Furthermore, exploration is often formulated as a local goal-seeking problem, neglecting spatial dependencies among unexplored regions at multiple scales and resulting in myopic behaviors with limited long-term planning capability [12]. Although graph neural networks (GNNs) are increasingly adopted to model spatial relationships [13]–[15], most GNN-based methods prioritize instantaneous graph states and ignore the temporal attributes of exploration. The accumulated exploration knowledge encoded within historical graph states is critical for guiding future decisions in long-horizon navigation [16]. Consequently, the effective integration of both spatial and temporal context is essential for DRL-based exploration to mitigate local optima under partial observability.

To address these challenges, we propose **SEEKER**, a novel deep reinforcement learning framework with **StructurE**-aware and **Knowledge-Evolving** spatiotempo**R**al fusion for robot exploration, which jointly encodes the structural properties of exploration regions and the evolution of historical knowledge on a sparse topological graph to enable sustained, non-myopic decision-making. The principal contributions of this paper are summarized as follows:

- 1) A structural descriptor is designed based on an encroaching strategy to quantify regional structural properties. By combining geometric scale, topological connectivity, boundary morphology, and structural evolution, the descriptor is incorporated into node features and encoded over the topology-derived graph by a GNN to yield compact spatial features.
- 2) An incremental Koopman operator-based embedding mechanism is developed to encode exploration-knowledge evolution on the expanding graph. A Multi-EMA refinement is proposed to consolidate the temporal states of persistent nodes, while an inheritance mechanism initializes the temporal embeddings of newly added nodes from adjacent regions.
- 3) An attention-integrated DRL policy fuses spatial and temporal features for non-myopic decision-making, while a dual-stage planner converts the selected target into locally executable and globally guided exploration paths. Extensive simulations and real-world experiments demonstrate the effectiveness and practical feasibility of

the proposed framework.

The remainder of this paper is organized as follows. Sec. II reviews related work. Sec. III presents the problem statement and system overview. Sec. IV details the proposed methodology. Sec. V presents the simulation and real-world experimental results. Sec. VII concludes the paper and discusses future directions.

II. RELATED WORK

A. Rule-based Exploration

The concept of “frontier” was first introduced by Yamauchi in [17] for robot exploration tasks. Since then, rule-based methods have dominated the field of autonomous exploration for decades. Beyond numerous frontier-based extensions designed to improve exploration efficiency [18]–[20], sampling-based approaches have emerged as a parallel paradigm [7], [8], [21]. These methods discretize traversable space by randomly generating candidate viewpoints and score each sample using handcrafted utility functions to select exploration targets. More recently, information gain-based methods [5], [22] have been proposed, which leverage entropy and statistical principles to enhance environment coverage while reducing computational overhead.

Despite this progress, rule-based exploration methods share several inherent limitations. The decision-making is often greedy toward unobserved regions, leading to redundant revisits and inefficient path planning [5]. Moreover, exploration robustness remains limited in dynamically evolving environments, with algorithms often effective only within their designed scenarios [9]. Furthermore, a fundamental trade-off exists between algorithmic complexity and exploration performance: a more comprehensive manual utility design may improve performance, but at the cost of increased computational complexity and degraded real-time responsiveness [6].

B. Learning-based Exploration

Learning-based methods have demonstrated greater potential than conventional approaches in addressing the trade-off between exploration and safety, and in learning effective navigation policies in unknown and partially observable environments [1]. Generally, learning architectures are categorized into two paradigms: end-to-end and modular.

End-to-end approaches train a unified policy that directly maps raw sensor observations to control commands, bypassing manually designed intermediate components. For instance, some studies integrate curriculum learning with deep reinforcement learning to improve sampling efficiency and adaptability across environments of varying scales [23], while others formulate object-oriented or goal-driven exploration policies that jointly couple perception and navigation within a single learning framework [24]. More recently, end-to-end DRL architectures have also been explored for tightly coupled exploration and task execution, such as jointly searching for and actively tracking targets using vision-based policies [25]. Despite these advances, end-to-end methods typically require carefully designed reward functions, millions of simulation steps, or extensive expert datasets,

and their sim-to-real transfer capability is often limited by insufficient generalization and safety guarantees [1]. Consequently, their application has largely remained within simulation environments.

Modular approaches comprise several distinct components, typically organized as a conventional pipeline consisting of a SLAM module for mapping, a frontier planner for navigation decision-making, and a low-level controller for path tracking [16]. Recent learning-based frameworks have further integrated DRL into this pipeline, with some replacing handcrafted planners with GNNs that leverage attention mechanisms or topological abstractions to reason over large-scale environments [11], [15], [26]. Compared with end-to-end paradigms, modular approaches are more widely adopted due to their interpretability, flexibility, and the ability to update individual modules without retraining the entire system [1]. Nevertheless, the performance of modular learning-based methods remains constrained by several unresolved issues. Although these graph-based planners retain topological connectivity of space, their node features typically lack explicit structural descriptors, failing to incorporate structured environmental information into the decision-making process. This prevents the policy from distinguishing structurally distinct regions and reasoning about traversal risk or exploration priority [27]. Additionally, some works overlook multi-scale spatial dependencies, which may induce myopic behaviors detrimental to long-term planning. Moreover, temporal attributes of the exploration process are often neglected, preventing accumulated exploration knowledge from effectively guiding future decision-making and leading to unnecessary revisits of explored areas [1].

Motivated by these observations, we propose a structure-enhanced and Koopman-integrated DRL framework as an alternative to traditional decision-making, addressing the aforementioned limitations in a unified manner. Specifically, a fine-grained structural descriptor is incorporated into node features, which are then embedded through a GNN, endowing the policy with explicit awareness of environmental structure. Meanwhile, historical exploration knowledge within a fixed temporal horizon is encoded via a Koopman operator, enabling efficient long-horizon temporal reasoning. These spatial representations and temporal embeddings are integrated through attention mechanisms to capture multi-scale spatial dependencies and maintaining logically consistent exploration policies, thereby reducing myopic behaviors and unnecessary revisits for efficient long-horizon decision-making.

III. PROBLEM STATEMENT

Consider a bounded 2D environment $\mathcal{M}_{\text{env}} \subseteq \mathbb{R}^2$, partitioned into the unknown region $\mathcal{M}_{\text{unknown}}$, the explored region $\mathcal{M}_{\text{free}}$, the obstacle region $\mathcal{M}_{\text{occ}}$, and the unreachable region $\mathcal{M}_{\text{unrc}}$. The robot exploration task is defined as follows: at each planning cycle, the robot generates a safe, collision-free exploration path L and incrementally updates its perceived map $\mathcal{M}_{\text{rob}}$ based on onboard observations (3D-LiDAR in this work), with the objective of maximizing coverage of $\mathcal{M}_{\text{free}}$ while minimizing path length or exploration time. Exploration

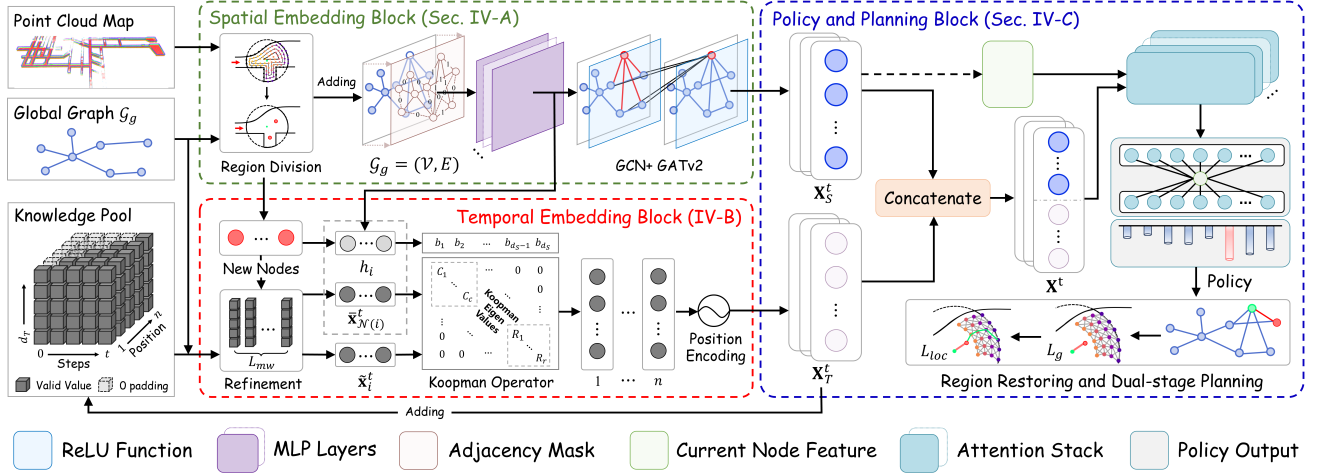


Fig. 1. Overview of the proposed **SEEKER** framework with three individual blocks: (Sec. IV-A) Node Spatial Embedding Block; (Sec. IV-B) Node Temporal Embedding Block; (Sec. IV-C) Policy and Planning Block.

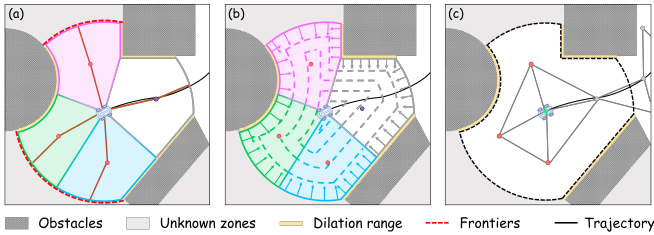


Fig. 2. Schematic of region division and structural encroaching. (a) Divided regions and beacon points based on skeleton. (b) ν_c and structural encroaching layers. (c) Expansion of $\mathcal{G}_g$.

is deemed complete when $\mathcal{M}_{\text{unknown}} = \emptyset$ and $\mathcal{M}_{\text{env}} \setminus \mathcal{M}_{\text{unrc}} = \mathcal{M}_{\text{rob}} = \mathcal{M}_{\text{free}} \cup \mathcal{M}_{\text{occ}}$.

The exploration problem is formulated as a Partially Observable Markov Decision Process (POMDP) defined by the tuple $\langle \mathcal{S}, \mathcal{A}, \mathcal{T}, \mathcal{O}, \Omega, \mathcal{R}, \gamma \rangle$, where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the discrete action set, $\mathcal{T}$ is the transition function, $\mathcal{O}$ is the observation function, Ω is the partial observations, $\mathcal{R}$ is the reward function, and $\gamma \in [0, 1)$ is the discount factor. At each step t , the robot chooses and takes an action $a_t \sim \pi(\cdot|o_t)$ at s_t , which indicates the navigation target, based on the observation o_t . Upon reaching the next state s_{t+1} with $\mathcal{T}(s_{t+1}|s_t, a_t)$, the agent receives a reward $r_t \in \mathcal{R}$. The objective is to find an optimal policy π^* , which selects actions at each decision step that maximizes the long-term discounted return $\sum_{t=0}^{\infty} \gamma^t r_t$.

IV. METHODOLOGY

A. Spatial Embedding Block

After the robot receives a new observation, the field of view (FOV) is considered within a radius of r_{max} . Due to obstacle occlusions, the visible area may deviate from an ideal circle, as illustrated in Fig. 2(a). We define the closed contour composed of the visible boundary and obstacle boundaries as the visibility contour ν_c , with length Γ_{ν_c} . To characterize potential exploration directions with different structural properties, we construct a sparse global graph $\mathcal{G}_g = (\mathcal{V}, E) \in \mathcal{M}_{\text{free}}$, where each vertex $v \in \mathcal{V}$ serves as a **beacon point** representing a structural region $\mathcal{R}_i$, and E denotes the set of collision-free edges connecting these vertices.

1) Global Graph Construction: Inspired by [28], we first extract the Hamilton-Jacobi skeleton from the Euclidean Signed Distance Field (ESDF) $D(\cdot)$ to capture the free-space topology (Fig. 2(a)). The skeleton is defined by positions with non-unique gradients in $D(\cdot)$ and forms the medial axis of the environment, characterizing traversable passages and connectivity. Based on the skeleton topology, nodes are categorized according to their branch degrees B_D : terminal nodes ($B_D = 1$), corridor nodes ($B_D = 2$), and junction nodes ($B_D \geq 3$). With junction nodes serving as structural delimiters, the skeleton is decomposed into consistent segments, and free-space points are assigned to their nearest skeleton segments to form topology-consistent regions. Each segmented region is represented by a beacon point from its corresponding skeleton segment. The region decomposition and beacon points are illustrated in Fig. 2(a).

Building on these topology-derived regions, we propose a structural encroaching method to quantify regional structural characteristics. The ν_c is first smoothed using a Gaussian filter to suppress local discretization noise and ensure contour continuity. Then, the tangent $\mathbf{t}$ and normal $\mathbf{p}$ vectors can be computed as:

$$\mathbf{n}_i = \frac{R_{\frac{\pi}{2}}(\mathbf{t}_i)}{\|R_{\frac{\pi}{2}}(\mathbf{t}_i)\|}, \quad \mathbf{t}_i = \frac{(\mathbf{p}_{i+1} - \mathbf{p}_i)}{\Delta s}, \quad i = 1, \dots, \lambda - 1 \quad (1)$$

where $|\Delta s| = \Gamma_{\nu_c}/\lambda$ is the sampling interval along the curve. $R_{\frac{\pi}{2}}$ stands for rotating $\mathbf{t}$ by $\pi/2$ toward the robot side. Then, the structural encroaching process is executed by iteratively displacing ν_c along $\mathbf{n}$ by a step distance $l = 1.8m$:

$$\nu_c^{(i)} = \nu_c - i \cdot l \cdot \mathbf{n}, \quad i = 1, \dots, N \quad (2)$$

until reaching the iteration limit (20 in this work) or self-intersection, with the maximum layer index recorded as N . The structural encroaching layers are illustrated in Fig. 2(b). In the following, we use $\nu_c^{(i)}|_{i=1}^N$ as an abbreviation for $\nu_c^{(i)}(s)$, $i = 1, \dots, N$.

Each vertex v contains the spatial coordinates (x, y) and three additional properties: information utility u , activation indicator f , and structural descriptor $\mathbf{G}$. The utility u quantifies observable frontiers from v , while $f = 1$ indicates

that the corresponding region is adjacent to frontiers and remains unexplored. To provide the policy with a compact yet complementary description of regional structure, the descriptor is designed considering four aspects: geometric scale, topological connectivity, boundary morphology, and structural evolution. These aspects are selected to capture general geometric and topological primitives of exploration regions rather than environment-specific statistics:

- **Geometric Scale** $G_s \in \mathbb{R}^3$: Derived from ESDF, including the maximum width (W_{max}), mean width (W_{mean}), and width variance (W_{var}), which characterize the space traversability and its geometric variability.
- **Topological Structure** $T_c \in \mathbb{R}$: Extracted from the skeleton using B_D , which reflects local connectivity and the associated decision complexity.
- **Boundary Morphology** $B_m \in \mathbb{R}^2$: Obtained from ν_c , considering the boundary length (Γ_{ν_c}) and curvature energy ($E_\kappa = \int_{\nu_c} (|\kappa| + \kappa(s)^2) ds$), which characterize the extent and geometric complexity of the visible boundary.
- **Structural Evolution** $S_e \in \mathbb{R}^3$: Derived from $\nu_c^{(i)}|_{i=1}^N$, encompassing the mean, variance, and maximum of the encroaching interlayer length variations to describe spatial trends during exploration.

The complete structural descriptor is obtained by concatenating four components as $\mathbf{G} = [G_s|T_c|B_m|S_e] \in \mathbb{R}^9$, providing a compact representation based on aggregated geometric and topological statistics rather than raw map-level features. During each planning cycle, newly extracted beacon points and the robot pose are added to $\mathcal{G}_g$, while inactive vertices are updated ($f = 0$) and invalid edges are removed to preserve graph sparsity.

2) **Graph Spatial Embedding**: The node features, including the proposed structural descriptor $\mathbf{G}$, are encoded over the global graph $\mathcal{G}_g$ using a GNN architecture. Specifically, we utilize a cascaded architecture consisting of a single-layer GCN [29] followed by multiple layers of GATv2 [30] for graph embedding. The GCN primarily provides an initial structural smoothing and a neighborhood-based representation for feature normalization, preventing premature overemphasis on individual node noise. GCN is written as:

$$\mathbf{H}^{(1)} = \sigma(\hat{\mathbf{D}}^{-\frac{1}{2}} \hat{\mathbf{A}} \hat{\mathbf{D}}^{-\frac{1}{2}} \mathbf{H}^{(0)} \mathbf{W}) \quad (3)$$

where $\mathbf{H}^{(0)} = [h_1, h_2, \dots, h_n]^T$ with $h_i \in \mathbb{R}^{ds}$ denotes the initial feature of node v_i obtained by an MLP. n is the number of nodes in $\mathcal{G}_g$ at step t . $\hat{\mathbf{A}} = \mathbf{A} + \mathbf{I}$ is the adjacency matrix with self-loops, and $\hat{\mathbf{D}}$ is its degree matrix. σ is the ReLU activation function, $\mathbf{W}$ is the weight matrix. The obtained node features are then processed by stacked GATv2 layers (4 in practice), where the attention mechanism adaptively aggregates informative features from distant nodes. The formulation of GATv2 is given by:

$$e_{ij} = \text{LeakyReLU}(\mathbf{a}^T (\mathbf{W}[h_i^{(l)} || h_j^{(l)}])) \quad (4)$$

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k \in \mathcal{N}(i)} \exp(e_{ik})} \quad (5)$$

$$h_i^{(l+1)} = \sigma(\sum_{j \in \mathcal{N}(i)} \alpha_{ij} \mathbf{W} h_j^{(l)}) \quad (6)$$

where $\mathcal{N}(i)$ is the neighbor set of node v_i . $\mathbf{a}$ is the attention vector that calculates the importance of node v_j to v_i . α_{ij} is the weight of each edge connecting to v_i . Finally, the spatial embedding $\mathbf{X}_S^t \in \mathbb{R}^{n \times ds}$ is obtained as:

$$\mathbf{X}_S^t = \sigma(\mathbf{H}_{\text{GATv2}}(\sigma(\mathbf{H}_{\text{GCN}}(\mathbf{H}^{(0)})))) \quad (7)$$

B. Temporal Embedding Block

To capture the temporal evolution of the exploration process, we propose a novel Koopman operator-based embedding method. The Koopman operator $\mathcal{K}$ characterizes nonlinear temporal dynamics through linear transformation in a high-dimensional lifted space $\mathbb{R}^{\mathcal{H}}$, formulated as $\mathbf{x}^t = \mathcal{K} \mathbf{x}^{t-1}$, where $\mathbf{x}$ denotes the latent temporal embedding. Importantly, $\mathcal{K}$ models the evolution of latent exploration states rather than complete environmental dynamics, as latent exploration states exhibit stronger temporal consistency during incremental exploration. This enables a fixed-dimensional linear operator to maintain temporal consistency over long-horizon exploration [31].

1) **Temporal Knowledge Update**: We maintain a knowledge pool K_p to store temporal embeddings from step 0 to $t-1$ as a 3D tensor (Fig. 1). The first dimension organizes graph nodes according to their entry times using sinusoidal positional encoding; the second dimension records the historical embeddings of each node across time steps; and the third dimension corresponds to the embedding dimension d_T . This structure preserves node identity and temporal order, enabling consistent node-wise temporal refinement over the exploration horizon. As the exploration graph $\mathcal{G}_g$ continuously expands when new structural regions are discovered, we design a two-step process for temporal embeddings of existing and newly added nodes: 1) existing nodes $\mathcal{V}_T^{t-1}$ are refined using their historical embeddings in K_p , and 2) newly added nodes $\Delta \mathcal{V}_T^t$ are initialized by inheriting temporal information from neighboring regions.

Specifically, for each existing node $v_i^t \in \mathcal{V}_T^{t-1}$, we perform a refinement using its historical temporal embedding states in the knowledge pool within a fixed sliding window $[t - L_{mw} + 1, t - 1]$ via a training-free Multi-EMA mechanism:

$$\tilde{\mathbf{x}}_i^t = \sum_{k=1}^{L_{mw}} \beta_k \mathbf{x}_i^{t-k}, \quad i = 1, \dots, n - \Delta n \quad (8)$$

$$\beta_k = \frac{\exp(-\eta k)}{\sum_{j=1}^{L_{mw}} \exp(-\eta j)} \quad (9)$$

where $\tilde{\mathbf{x}}_i^t \in \mathbb{R}^{d_T}$ denotes the refined temporal embedding, $\eta = 0.5$ is the decay coefficient, and Δn represents the number of newly added nodes. When the available history length of a node is shorter than L_{mw} , the missing entries are zero-padded at the front of the window. The exponential decay weighting in Multi-EMA captures the temporal evolution of regional exploration states, enabling the policy to distinguish exploration progress across regions.

For newly incorporated nodes $v_i^t \in \Delta \mathcal{V}_T^t$, we aggregate the refined embeddings of spatial neighbors to form an inheritance embedding $\bar{\mathbf{x}}_{\mathcal{N}(i)}^t \in \mathbb{R}^{d_T}$. This design is based on the observation that exploration maturity exhibits spatial

continuity in $\mathcal{G}_g$: neighboring regions generally undergo correlated exploration processes, and their refined embeddings already encode historical exploration states.

$$\tilde{\mathbf{x}}_{\mathcal{N}(i)}^t = \frac{1}{|\mathcal{N}(i)|} \sum_{k \in \mathcal{N}(i)} \tilde{\mathbf{x}}_k^t, \quad i = 1, \dots, \Delta n \quad (10)$$

Therefore, the inherited embedding provides an estimation of the temporal state for newly observed regions before sufficient historical observations are accumulated.

2) Graph Temporal Embedding: The refined and initialized temporal states are subsequently propagated using a quasi-diagonal Koopman operator, where the eigenvalues of state matrices are identified directly to match the spectral decomposition form and reduce redundant parameters [31]. The state matrices $\mathbf{A} \in \mathbb{R}^{d_T \times d_T}$ and $\mathbf{B} \in \mathbb{R}^{d_T \times d_S}$ are defined as:

$$\mathbf{A} = \text{diag}\{\mathbf{C}_1, \dots, \mathbf{C}_c, R_1, \dots, R_r\} \quad (11)$$

$$\mathbf{B} = [\mathbf{b}_1, \mathbf{b}_2, \dots, \mathbf{b}_{d_S}] \quad (12)$$

where $\mathbf{C}_j = [a_j, b_j; b_j, -a_j] \in \mathbb{R}^{2 \times 2}$ represents complex conjugate eigenvalues and $R_i \in \mathbb{R}$ denotes real eigenvalues, such that $c + r = d_T$. In this configuration, the Koopman operator is written in an equivalent form as $\mathcal{K} = [\mathbf{A}, \mathbf{B}]$. Following [31], we set $r = d_T \% 2$ and $c = (d_T - r)/2$, with block entries of $\mathbf{A}$ drawn from a Gaussian scaled by d_T and $\mathbf{B}$ using default bias-free linear initialization. Therefore, the temporal embedding $\mathbf{X}_T^t = [\mathbf{x}_1^t, \dots, \mathbf{x}_n^t]^\top \in \mathbb{R}^{n \times d_T}$ can be updated according to Eq. (13) and subsequently stored in the knowledge pool:

$$\mathbf{x}_i^t = \begin{cases} \mathbf{A}\tilde{\mathbf{x}}_i^t, & \text{if } v_i^t \in \mathcal{V}_T^{t-1} \\ \mathbf{A}\tilde{\mathbf{x}}_{\mathcal{N}(i)}^t + \mathbf{B}h_i, & \text{else } v_i^t \in \Delta\mathcal{V}_T^t \end{cases} \quad (13)$$

where h_i denotes the output from the MLP within the spatial embedding module.

C. Policy and Planning Block

1) Policy Output: The spatial embeddings $\mathbf{X}_S^t$ and temporal embeddings $\mathbf{X}_T^t$ are concatenated to form the unified state feature $\mathbf{X}^t = [\mathbf{X}_S^t | \mathbf{X}_T^t] \in \mathbb{R}^{n \times (d_S + d_T)}$, where the feature corresponding to the robot's current position p^t is selected as h_c . Then, h_c is iteratively refined through a stack of cross-attention layers that selectively aggregates graph global contextual information from $\mathbf{X}^t$ to support long-horizon decision-making. The attention layer reads:

$$q_i = W_Q h_i^q, \quad k_i = W_K h_i^{k,v}, \quad v_i = W_V h_i^{k,v}, \quad u_{ij} = \frac{q_i^T \cdot k_j}{\sqrt{d_S + d_T}}$$

$$w_{ij} = \begin{cases} \frac{\exp(u_{ij})}{\sum_{j=1}^n \exp(u_{ij})}, & M_{ij} = 0 \\ 0, & M_{ij} = 1 \end{cases}, \quad h'_c = \sum_{j=1}^n w_{ij} v_j \quad (14)$$

where q, k, v denote the source of the *query*, *key*, and *value* respectively. $W_Q, W_K, W_V \in \mathbb{R}^{(d_S + d_T) \times (d_S + d_T)}$ are learnable matrices. M is an adjacency matrix built from the edge set E of $\mathcal{G}_g$, and is used as an edge mask.

Specifically, at each attention layer, h_c serves as the query, while the features of all other nodes in $\mathcal{G}_g$ are used as keys and values. After L layers (4 in practice) of attention-based refinement, the output $h_c^{(L)}$ serves as the final current state representation and is passed to a pointer layer [15], where

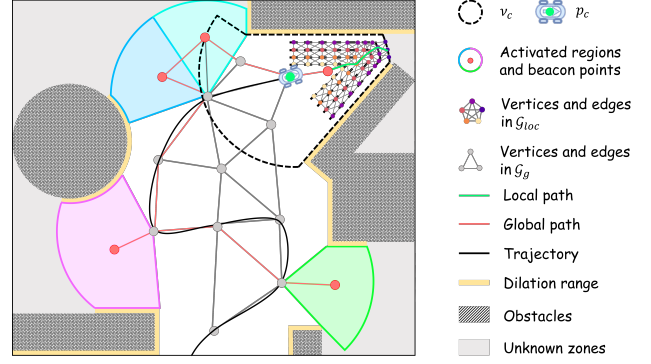


Fig. 3. Schematic of dual-stage planning.

neighboring nodes with $f = 1$ (region activated) are treated as candidate actions. By computing attention scores between $h_c^{(L)}$ and the embeddings of these valid neighboring nodes, the pointer layer generates a categorical distribution over feasible actions, yielding the policy $\pi(a_t | o_t) = p(v_m | o_t)$, $\forall (p^t, v_m) \in E$. This design naturally aligns the policy output dimension with the local graph connectivity, enabling adaptive action selection for autonomous exploration. The final selected target beacon point and its associated region are defined as v_{tgt} and $\mathcal{R}_{tgt}$. The policy network structure can be found in Fig. 1.

2) Dual-stage Planning: Inspired by [7], we develop a dual-stage planning scheme consisting of a local exploration path toward v_{tgt} and a global guidance path when the current target becomes unreachable. Specifically, the encroaching layers and visibility contour $v_c \in \mathcal{M}_{\text{free}}$ are first restored within $\mathcal{R}_{tgt}$. A curvature-aware sampling strategy is then performed along each encroaching layer, where the sampling interval is adaptively determined by:

$$\Delta s = \frac{\iota_0}{1 + |\kappa(\iota)|} \quad (15)$$

where ι_0 is the predefined sampling resolution. The resulting candidates form the local node set $\mathcal{V}_{\text{loc}}$. Compared with uniform sampling, this strategy allocates denser samples to geometrically complex regions while reducing redundant nodes in regular areas.

Subsequently, a dense local topological graph $\mathcal{G}_{\text{loc}} = (\mathcal{V}_{\text{loc}}, E_{\text{loc}})$ is constructed by organizing sampled nodes according to their corresponding encroaching layers, while excluding layers whose candidate points all deviate from the frontier direction, as depicted in Fig. 3. The candidate node set of the i -th layer is: $\mathcal{V}_{\text{loc},i} = \{v \in \mathcal{R}_{tgt} \mid v \in v_c^{(i)}, i = 1, \dots, M\}$. The local exploration path L_{loc} is obtained by selecting one optimal candidate from each layer in reverse order, from the outermost layer M to the innermost layer 1:

$$v_i^* = \arg \max_{v \in \mathcal{V}_{\text{loc},i}} \omega_1 u(v) + \omega_2 \cos(\theta_v) - \omega_3 d(v) + \omega_4 D(v)$$

$$\text{s.t. } v_{M+1}^* = v_{tgt}$$

$$\theta_v = \begin{cases} \angle(p \rightarrow v_{tgt}, v_{tgt} \rightarrow v), & \text{if } i = M \\ \angle(v_{i+2}^* \rightarrow v_{i+1}^*, v_{i+1}^* \rightarrow v), & \text{else} \end{cases} \quad (16)$$

where $u(v)$ is the information utility and $v_m \rightarrow v_n$ denotes the vector from v_m to v_n . The angular similarity $\cos(\theta_v)$ and distance penalty $d(v) = |v - v_{i+1}^*|$ encourage smooth transitions and mitigate zigzag motions, while the ESDF

clearance $D(v)$ keeps the path near the region's center. Four terms in 16 are normalized and weighted by $\omega_1\text{--}\omega_4$, which are fixed as $[1.0, 0.5, 0.2, 0.4]$ across scenarios. Since the number of encroaching layers is adaptively determined by the local region structure, the same candidate-selection criterion can be independently applied to each layer. Consequently, the local path optimization requires only a linear scan over all candidates, with a complexity of $O(|\mathcal{V}_{\text{loc}}|)$. L_{loc} is formed by the node sequence $\{p^t, v_{\text{tgt}}, v_M^*, \dots, v_1^*\}$ with adjacent nodes connected through E_{loc} .

Finally, based on the connectivity of $\mathcal{G}_g$, the global path L_g is generated by solving an ATSP [18] over other activated regions, providing fallback guidance when the current target becomes unreachable or local exploration fails to generate new beacon points, thereby maintaining exploration continuity.

3) Training Details: We train our exploration policy using the discrete Soft Actor-Critic (SAC) algorithm [32], which aims to maximize a trade-off between the expected return and the policy entropy:

$$\pi_\theta^* = \operatorname{argmax}_{\pi} \sum_{t=0}^T \mathbb{E} [\gamma^t (r_t + \alpha \mathcal{H}(\pi(\cdot|o_t)))] \quad (17)$$

Here, the policy output $\pi(\cdot|o_t)$ is a categorical distribution over the discrete action set $\mathcal{A}_t$, and each $\pi(a|o_t) \in [0, 1]$ denotes a probability rather than density [32]. $\mathcal{H}(\pi(\cdot|o_t)) = -\sum_a \pi(a|o_t) \log(\pi(a|o_t))$ denotes the policy entropy that encourages diverse action exploration. α is the temperature parameter. To provide dense guidance, the action reward r_t is designed as a multi-component signal $r_t = \omega_c r_c + \omega_u r_u + \omega_s r_s + r_f$. Apart from the movement cost $r_c = -\|p^{t+1} - p^t\|_2$ and the exploration reward r_u (quantity of observable frontiers when taking action a_t), we introduce a structural reward r_s to prioritize spatial discovery. This term assigns sparse bonuses for first-time visits to key regions: entering activated regions (3.0), reaching intersections (2.0), advancing through corridors (0.3), and arriving at v_{tgt} (2.0). When sufficient exploration completion is achieved ($\mathcal{M}_{\text{unknown}} = \emptyset$ or the maximum node utility falls below 5), a finishing reward $r_f = 20$ is given to the agent. $\omega_{c,u,s}$ are scaling factors for each reward component and set to $[5, 1, 0.8]$ in practice. SAC trains dual critic networks $\phi_{1,2}$ to estimate the soft action-state values $Q_\phi(o_t, a_t)$ by:

$$J_Q(\phi) = \mathbb{E}_{o_t} \left[\frac{1}{2} (Q_\phi(o_t, a_t) - (r_t + \gamma \mathbb{E}_{o_{t+1}} [V(o_{t+1})]))^2 \right] \quad (18)$$

where $V(o_t) = \sum_a \pi(a|o_t) [Q_\phi(o_t, a) - \alpha \log(\pi(a|o_t))]$. The policy network θ is trained to maximize the expected state-action value, and α is auto-tuned during training. The policy and temperature losses are defined as:

$$J_\pi(\theta) = \sum_a \pi(a|o_t) [\alpha \log(\pi_\theta(a|o_t)) - Q_\phi(o_t, a)] \quad (19)$$

$$J(\alpha) = \sum_a \pi(a|o_t) [-\alpha \log(\pi(a|o_t)) + \bar{\mathcal{H}}] \quad (20)$$

where $\bar{\mathcal{H}}$ is the target entropy and set to $-0.1 \ln \frac{1}{K}$ in practice to prevent entropy collapse and maintain exploration diversity.

In addition, to enforce temporal consistency of the Koopman propagation, we design a dynamics consistency loss that penalizes the one-step prediction error of the lifted state and provides a self-supervised temporal regularization:

Algorithm 1 SEEKER Training Procedure.

Require: Maximum episode duration N_{max} , batch size B_s , warmup N_{warm} , policy π_θ , critics Q_{ϕ_1}, Q_{ϕ_2} , temperature α , replay buffer $\mathcal{B}$, knowledge pool K_p

- 1: Init $\theta, \phi, \alpha; \mathcal{B} \leftarrow \emptyset$
- 2: **while** training **do**
- 3: Reset; $\mathcal{G} \leftarrow \emptyset; K_p \leftarrow \emptyset$
- 4: **for** $t = 0, \dots, N_{\text{max}} - 1$ **do**
- 5: $\mathcal{G} \leftarrow \text{ExpandGraph}(\mathcal{G}, o_t)$
- 6: $\{o_t, g_t\} \leftarrow \text{GetObservation}(\mathcal{G})$
- 7: $\mathbf{X}_S^t \leftarrow \text{SpatialEmbed}(o_t)$ // Eq. (7)
- 8: $\mathbf{X}_T^t, K_p \leftarrow \text{TemporalEmbed}(\mathbf{X}_S^t, K_p)$ // Eq. (13)
- 9: $a_t \sim \pi_\theta(\cdot | o_t)$
- 10: $\{r_t, o_{t+1}, g_{t+1}, d_t\} \leftarrow \text{EnvStep}(a_t)$
- 11: $\mathcal{B} \leftarrow \mathcal{B} \cup \{(o_t, g_t, a_t, r_t, o_{t+1}, g_{t+1}, d_t, t, \text{coords})\}$
- 12: **if** $|\mathcal{B}| \geq N_{\text{warm}}$ **then**
- 13: $\{J_Q, J_\pi, J(\alpha), J_K\} \leftarrow \text{Replay}(\mathcal{B}, B_s; \phi, \theta, \alpha)$
- 14: **end if**
- 15: **Exploration finish ?** break : continue
- 16: **end for**
- 17: **end while**

$$J_K = \frac{1}{|\mathcal{V}_T^{t-1}|} \sum_{i: v_i^t \in \mathcal{V}_T^{t-1}} \|\mathbf{x}_i^t - \mathbf{A}\mathbf{x}_i^{t-1}\|_2^2 \quad (21)$$

Our model is trained on a dungeon environment dataset provided by [15], where each environment is a 640×480 2D grid map with a resolution of $0.4m$, and the sensor range is set to $r_{\text{max}} = 16m$. Regarding the architectural configurations, we specify the spatial and temporal embedding dimensions as $d_S = 128$ and $d_T = 64$, respectively, with a temporal sliding window of $L_{mw} = 8$. For training-related parameters, the maximum episode duration $N_{\text{max}} = 128$, the size of replay buffer $\mathcal{B} = 10000$, the warm-up threshold $N_{\text{warm}} = 2000$, and the batch size $B_s = 128$. We use the Adam optimizer with a learning rate of 10^{-5} for the actor/critic and 10^{-4} for temperature tuning, with a discount factor $\gamma = 0.95$. To enhance data throughput, we leverage the Ray distributed framework to execute 10 simulation workers in parallel. All training procedures are conducted on a workstation equipped with an AMD Ryzen 9 9950X3D CPU, an NVIDIA GeForce RTX 4080 Super GPU, and 64 GB of RAM. The policy typically achieves convergence within 24 hours. The training procedure is summarized in Algorithm 1.

V. EXPERIMENTS AND ANALYSIS

A. Simulation Experiments

To validate the effectiveness of the proposed framework, we conduct extensive simulations first in a comprehensive simulation environment built on **Gazebo** provided by [14] and [7], where the robot is a differential-drive platform equipped with a 16-line LiDAR, and the maximum velocity is limited to $2m/s$. We refer to [15] and employ Octomap to represent spatial free and occupied areas (derived from the fusion of point cloud data provided by SLAM model [33]), with its 2D projection map utilized for sparse graph construction. Three representative scenarios with varying structural complexities and scales are selected (Fig. 4), which are not included in the training set:

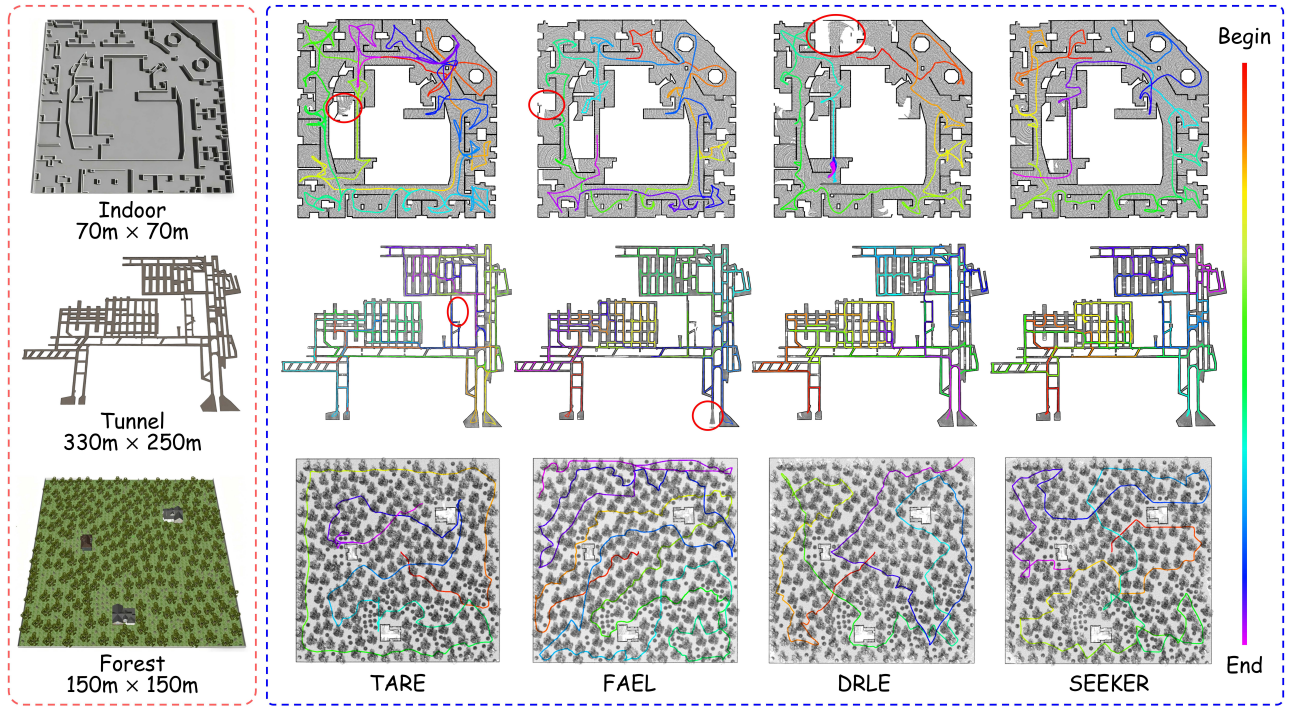


Fig. 4. Simulation environments and representative exploration results. **Left:** Top-down views of the three simulation scenarios (Indoor, Tunnel, and Forest). **Right:** Representative exploration trajectories and the corresponding point-cloud maps produced by each comparative method, where trajectories are rendered in a rainbow colormap to indicate temporal progresses coverage fromression, and **red ellipses** highlight missed regions during exploration.

- **Indoor** ($70m \times 70m$): A small-scale structured environment characterized by doorways and rooms.
- **Tunnel** ($330m \times 250m$): A large-scale structured scenario featuring numerous branches and confined tunnels.
- **Forest** ($150m \times 150m$): An unstructured environment filled with obstacles and irregular geometries.

We deploy SEEKER to the simulation environment through ROS1 and compare it against three benchmarks in different paradigms:

- **TARE** [7]: A dual-resolution sampling-based method with dense local map for exploration path planning and sparse global map for guidance path planning. We adapted the *local planning range* and *sampling density* according to the scene scale.
- **FAEL** [18]: A frontier-based approach that constructs a sparse roadmap to achieve heuristic exploration path planning in large environments. We mainly adjusted the *local map resolution* and the *frontier ignore threshold* to adapt to our simulation environment.
- **DRLE** [15]: A DRL-based method that leverages the attention mechanism on a sparse graph to sequentially choose navigation targets for robot exploration. We directly adopted the publicly available *pretrained model* under our simulation environment.

Other parameters for the benchmarks are kept consistent with their officially recommended configurations. All methods are evaluated over 10 independent trials in each of the three scenarios with random starting positions. The exploration is considered terminated when either the environment is sufficiently covered or the robot fails to make effective exploration progress, indicated by prolonged stationary behavior. Additionally, a maximum exploration time of 5000s

TABLE I
PERFORMANCE COMPARISON ACROSS THREE SCENARIOS

Scenarios / Size (m^2) / Max area ¹ (m^2)	Methods	Exploration Time (s)		Movement Distance (m)		Computation Time (s)		Coverage Rate ² (%)	
		Avg	Std	Avg	Std	Avg	Std	Avg	Std
Indoor 70m×70m 3500	TARE	770.67	63.46	1166.61	83.54	0.470	0.012	98.96	0.37
	FAEL	608.95	113.72	698.93	124.02	0.018	0.003	96.97	8.83
	DRLE	619.93	108.61	829.83	161.83	0.110	0.005	92.41	3.71
	SEEKER	592.39	62.08	723.74	37.59	0.072	0.004	99.12	0.76
Tunnel 330m×250m 22000	TARE	3086.88	291.33	5074.48	249.87	0.338	0.112	98.52	1.66
	FAEL	2949.01	236.50	4135.23	374.02	0.029	0.018	96.27	3.70
	DRLE†	3394.42	332.74	5308.95	206.36	0.275	0.063	98.35	3.24
	SEEKER	2890.81	227.56	3472.68	198.83	0.194	0.052	98.82	0.36
Forest 150m×150m 22300	TARE	768.23	66.70	1387.93	74.27	0.215	0.007	98.56	1.22
	FAEL	1083.82	172.31	1527.62	251.54	0.055	0.008	96.17	6.24
	DRLE	828.54	149.83	1332.95	284.64	0.231	0.212	96.48	1.96
	SEEKER	760.21	34.56	1323.71	83.98	0.151	0.029	98.89	0.74

¹Max area: $\mathcal{M}_{env} \setminus \mathcal{M}_{unrc}$, the maximum area that can be observed theoretically.

²Coverage rate: the ratio of actual explored area to max area when exploration stops.

†: higher is better. ↓: lower is better.

Bold: the best result. Underlined: the second-best result.

is imposed to prevent indefinite execution. A trial is deemed failed if the coverage rate at termination is below 90%. To quantitatively assess the performance of each method, we record four key metrics: exploration time, movement distance, computation time, and coverage rate over 10 independent trials, as summarized in Table I, where failure cases are explicitly marked with † and not included in the statistics. The most representative exploration trajectories and process curves across three scenarios are shown in Fig. 4 and 5.

1) **Environment Coverage:** Coverage rate measures the proportion of observable area explored upon termination. As shown in Table I, SEEKER achieves the highest coverage rate across all three scenarios (all above 98.8%) with smooth, minimally repetitive trajectories and few missed regions, as depicted in Fig. 4. TARE attains comparable coverage through exhaustive local coverage planning, but this comes at substantially higher time and distance costs (see Sec. V-A.2). FAEL exhibits lower coverage in the more complex Tunnel and Forest scenarios, trailing SEEKER by 2.6–2.7%.

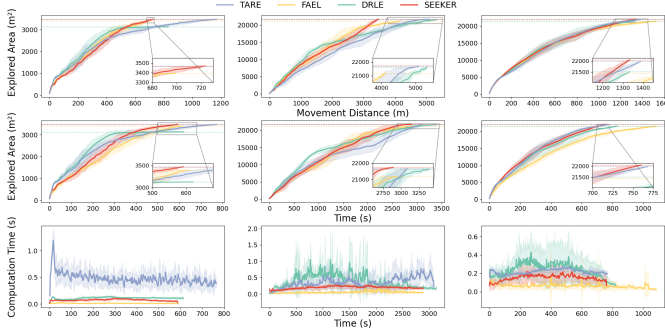


Fig. 5. Exploration process curves across three scenarios. **Row 1:** Explored area over movement distance (EA-MD). **Row 2:** Explored area over time (EA-T). **Row 3:** Per-iteration computation time over time (PC-T). Shaded regions indicate standard deviation across 10 trials. **Columns** from left to right: **Indoor**, **Tunnel**, **Forest**.

As highlighted by the red ellipses in Fig. 4, FAEL tends to miss isolated regions in narrow or topologically complex areas due to incomplete frontier detection. DRLE achieves relatively high coverage in Forest (96.48%) and successful Tunnel trials (98.35%), indicating the potential of learning-based exploration. However, its exploration behavior remains less reliable. In Indoor, the coverage drops to 92.41%, and failure trials are observed in Tunnel, where DRLE occasionally becomes trapped in narrow corridor structures and fails to generate further exploration progress. This behavior is consistent with the limitation of a history-agnostic attention mechanism, which may select incoherent target sequences, causing repeated retraversal and incomplete coverage. In contrast, SEEKER consistently maintains high coverage across all scenarios by fusing structure-enhanced spatial embeddings with exploration-knowledge-refined temporal embeddings to guide coherent exploration.

2) Exploration Efficiency: Exploration time and movement distance jointly characterize exploration efficiency. As reported in Table I, SEEKER achieves the lowest exploration time across all three scenarios, while obtaining the shortest movement distance in Tunnel and Forest and the second-shortest in Indoor, without sacrificing coverage. TARE incurs significant efficiency costs for its high coverage. The EA-MD and EA-T curves exhibit obvious slow-rising segments in Indoor and Tunnel, as shown in Fig. 5. This is because when the information gains of remaining subspaces are comparable, the planner repeatedly switches between them, traversing explored areas and inflating both time and distance. FAEL achieves the shortest movement distance in Indoor and the second-shortest in Tunnel, but its efficiency degrades in Forest. Its continued increase in EA-MD after SEEKER has converged indicates additional backtracking to recover previously overlooked enclosed areas, leading to prolonged exploration. DRLE exhibits uneven efficiency across scenarios: in Indoor, its movement distance exceeds both SEEKER and FAEL, reflecting path detours caused by missing structural awareness. In Tunnel, the longer movement distance reflects target-selection instability: the policy oscillates between competing frontier regions, leading to repeated corridor traversal. In contrast, SEEKER maintains a steady increase in both EA-MD and EA-T, avoiding the rapid accumulation or prolonged growth observed in the other

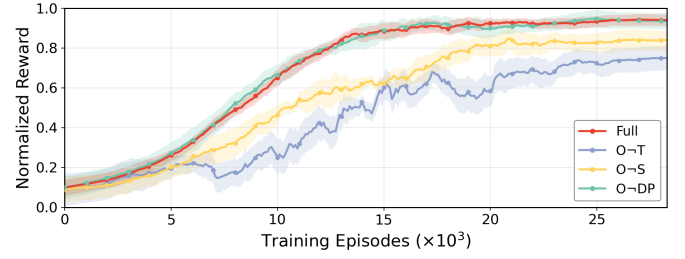


Fig. 6. Training curves for different ablation variants.

methods. On average, SEEKER reduces exploration time by 10.2%, 11.5%, 9.2% and movement distance by 24.7%, 8.6%, 16.0% compared to TARE, FAEL, and DRLE averaged across three scenarios. This efficiency advantage stems from the combination of structure-aware target selection and spatial-temporal representations, which enables more coherent long-horizon exploration, while the dual-stage planner further reduces unnecessary traversal through local path optimization and global topological guidance.

3) Algorithm Responsiveness: Computation time is an important indicator of real-time responsiveness. As reported in Table I, SEEKER maintains low computational costs across all three scenarios, with average runtimes of 0.072s, 0.194s, and 0.151s, while achieving higher coverage. As shown in Fig. 5 (PC-T), TARE incurs consistently high computational costs due to its dense local sampling and global path search, particularly in Indoor (0.470s). FAEL achieves the lowest runtime in all scenarios through lightweight planning on a sparse roadmap, but this computational advantage is accompanied by lower coverage, especially in complex environments. DRLE exhibits moderate runtime in Indoor (0.110s) and Forest (0.231s), but shows considerable fluctuations in Forest, where the standard deviation reaches 0.212s, indicating less stable computational behavior. In contrast, SEEKER maintains relatively stable computational costs throughout exploration. This efficiency is enabled by the compact search spaces of both planning stages: global guidance is solved on a sparse topological graph, while local path optimization considers only candidate points filtered by encroaching layers. In addition, the quasi-diagonal Koopman operator provides efficient latent-state propagation with low inference overhead. These designs allow SEEKER to maintain responsive planning while adapting its computational cost to the topological complexity of the environment.

B. Ablation Studies

To evaluate the contribution of each core component within SEEKER, we conduct ablation studies in the **Indoor** scenario. Three ablation variants are considered: removing the Koopman operator-based temporal embedding block, including Multi-EMA refinement and inheritance mechanism (O~T); removing the structural descriptor and GNN-based spatial embedding block (O~S); removing the dual-stage planning module, where the robot directly navigates to the selected target beacon point (O~DP). All variants retain the same reward formulation and are trained with three independent random seeds. The corresponding training curves are shown in Fig. 6. Quantitative results are summarized in Table II.

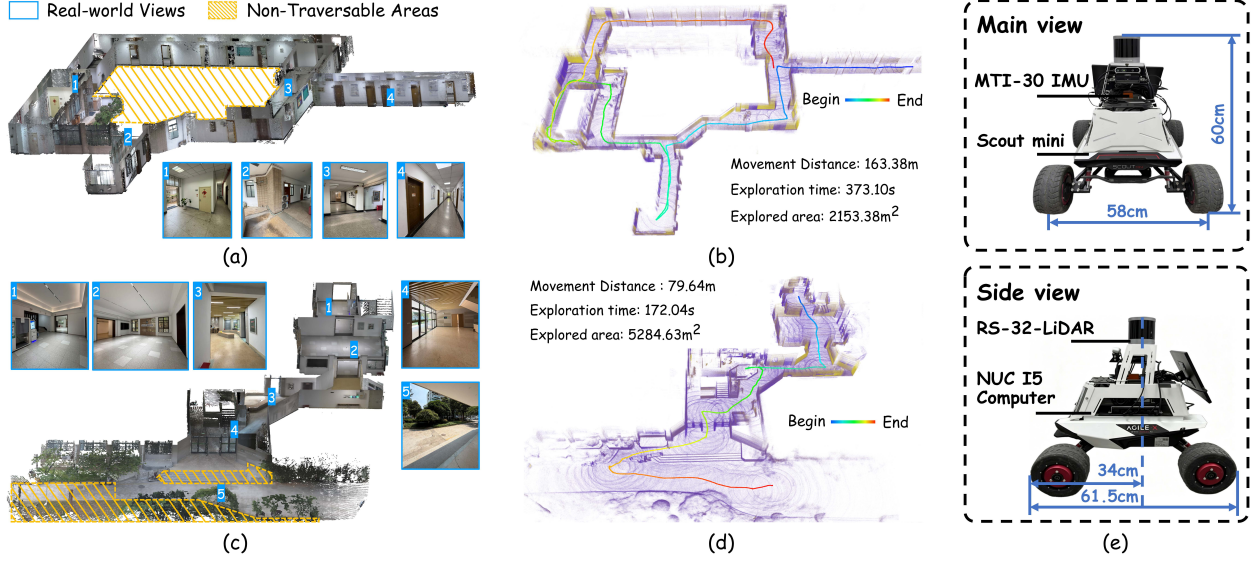


Fig. 7. Real-world experiment scenes and exploration results. (a, c) Schematics of **Loop** and **Hall** environments reconstructed using a handheld scanner, with representative real-world views and non-traversable areas annotated. (b, d) Exploration results, showing the trajectory and the constructed point cloud map. (e) The Scout Mini wheeled robot and onboard sensors. A demonstration video is provided as supplementary material.

TABLE II

QUANTITATIVE RESULTS OF ABLATION STUDIES IN INDOOR SCENARIO

Methods	Exploration Time (s) ↓		Movement Distance (m) ↓		Algorithm Runtime (s) ↓		Coverage Rate (%) ↑
	Avg	Std	Avg	Std	Avg	Std	
O→T	887.02	118.23	783.90	168.37	0.069	0.012	95.42
O→S	1006.06	133.40	771.92	59.44	0.061	0.017	93.51
O→DP	647.46	90.56	866.08	220.01	0.065	0.014	97.71
SEEKER	592.39	62.08	723.74	37.59	0.072	0.004	99.12

O→T reduces coverage from 99.12% to 95.42% and increases exploration time by 49.7%. Without historical knowledge refinement, the policy becomes more prone to revisiting previously explored regions, indicating that the proposed mechanism is critical for maintaining temporally consistent exploration knowledge during long-horizon decision-making. The training curve further shows a less stable learning process with lower final performance, demonstrating the benefit of the Koopman-based temporal representation in guiding policy learning over long-horizon exploration.

O→S produces the largest coverage degradation to 93.51%, with the exploration time and movement distance reaching 1006.06s and 771.92m, respectively. Its training curve shows a slower improvement trend and lower final performance than SEEKER, consistent with the degradation observed during deployment. Without the structural descriptor, the policy lacks explicit regional cues and the ability to distinguish regions with different geometric and topological characteristics, resulting in less complete coverage of isolated regions.

O→DP achieves a comparable coverage of 97.71%, but increases the movement distance by 19.7%. Its training curve closely follows that of SEEKER, suggesting that the performance difference mainly arises during deployment rather than policy learning. Without local path optimization and global guidance, direct navigation tends to produce geometrically direct but topologically inefficient routes, resulting in unnecessary movement. Notably, compared to O→DP, SEEKER achieves improved path efficiency while incurring only a marginal computational overhead, with the runtime increasing from 0.065s to 0.072s, indicating that

the dual-stage planning module provides substantial planning benefits at limited additional computational cost.

C. Real-world Experiments

To further validate the practical feasibility of SEEKER, field experiments were conducted in two indoor environments: 1) a loop-shaped corridor with intersections (**Loop**, Fig. 7(a)), and 2) a building entrance hall with varying spatial scales and structural features (**Hall**, Fig. 7(c)). The experiments were performed on a Scout Mini wheeled robot equipped with an onboard PC (i5-1340P CPU and 16 GB of RAM), an MTI-30 IMU, and an RS-LiDAR-32 (Fig. 7(e)). The terrain map in [18] was used for traversability analysis, where negative obstacles (e.g., stairs) were not considered in our implementation and blocked as non-traversable areas. The trained model was deployed in a zero-shot manner without additional tuning or retraining. All computations were performed onboard, with the maximum velocity limited to 0.6m/s for safety.

The exploration trajectories and constructed point cloud maps are illustrated in Fig. 7(b) and (d). In **Loop**, the robot completed exploration in 373.10s with a movement distance of 163.38m, covering 2153.38m². As shown in Fig. 7(b), the policy effectively handled intersections without excessive backtracking. In **Hall**, the robot explored 5284.63m² in 172.04s over 79.64m, covering all observed traversable regions, including those with rapid changes in spatial scale and structural features such as Region 3 and 4. Overall, the results demonstrate the practical feasibility, efficiency, and zero-shot capability of SEEKER for real-world exploration.

VI. DISCUSSION AND FUTURE WORK

Although the proposed framework demonstrates effective exploration performance, several challenges remain to be further addressed: 1) The current implementation is mainly designed for robots operating in 2D environments. Extending the framework to 3D volumetric exploration requires further investigation, particularly in adapting the spatial descriptor from 2D geometric structures (e.g., contours and skeletons)

to 3D counterparts. 2) The framework currently relies on geometric exploration cues without semantic understanding, which may limit its applicability to more complex real-world exploration tasks. Correspondingly, future work will focus on extending the proposed framework toward 3D autonomous exploration, integrating vision-language models for semantic-aware decision-making.

VII. CONCLUSION

In this work, we proposed SEEKER, a deep reinforcement learning framework with structure-aware and knowledge-evolving spatiotemporal fusion for robot exploration of unknown environments. Rather than treating spatial perception and temporal reasoning in isolation, SEEKER encodes the structural properties of exploration regions through skeleton-driven region division and structural encroaching, while capturing the evolution of historical exploration knowledge through a Koopman operator. An attention mechanism fuses these spatial and temporal representations to support non-myopic decision-making, and a dual-stage planning scheme further generates efficient local exploration and global guidance paths to ensure exploration continuity. Extensive simulations across three unseen scenarios demonstrate that SEEKER achieves superior coverage, exploration efficiency, and computational responsiveness against comparative methods. Real-world experiments further validate its practicality and potential for application in industrial intelligent systems.

REFERENCES

- [1] M. E. Moyo and T. Celik, "Learning to explore: A systematic review of learning-based single mav exploration in confined environments," *Arab. J. Sci. Eng.*, 2025.
- [2] X. Zhang, X. Xu, Y. Liu, H. Wang, X. Zhang, and Y. Zhuang, "FGIP: A frontier-guided informative planner for uav exploration and reconstruction," *IEEE Trans. Ind. Informat.*, vol. 20, no. 4, 2024.
- [3] Y. Jia, C. Wang, W. Tang, Z. Wang, and Z. Sun, "Unlocking full exploration potential of ground robots by multiresolution topological mapping," *IEEE Trans. Ind. Informat.*, vol. 21, no. 8, 2025.
- [4] X. Xu, X. Zhang, Y. Liu, H. Wang, X. Zhang, and Y. Zhuang, "Semantic-driven informed planning and 3D reconstruction for the quadrotor unmanned aerial vehicle," *IEEE Trans. Veh. Technol.*, vol. 74, no. 3, 2025.
- [5] Z. Wang, S. Pan, J. Xu, X. Tao, W. Gao, and Q. Wang, "TIPS: Tiered information-rich planning strategy for efficient agv autonomous exploration," *IEEE Robot. Automat. Lett.*, vol. 10, no. 12, pp. 12 764–12 771, 2025.
- [6] Z. Che, Y. Zhang, S. Liang, B. Zhou, J. Ma, and J. Zhou, "GUIDE: A diffusion-based autonomous robot exploration framework using global graph inference," 2025. [Online]. Available: <https://arxiv.org/abs/2509.19916>
- [7] C. Cao, H. Zhu, H. Choset, and J. Zhang, "TARE: A hierarchical framework for efficiently exploring complex 3D environments," in *Robot.: Sci. Syst.*, 2021.
- [8] B. Andreas, K. Mina, A. Kostas, O. Helen, and S. Roland, "Receding horizon "Next-Best-View" planner for 3D exploration," in *Proc. of the IEEE Intl. Conf. on Robot. and Autom. (ICRA)*, 2016, pp. 1462–1468.
- [9] S. Liao, X. Lv, J. Lew, S. Zhang, J. Liang, P. Li, Y. Cao, W. Wu, and G. Sartoretti, "FARE: Fast-slow agentic robotic exploration," 2026. [Online]. Available: <https://arxiv.org/abs/2601.14681>
- [10] Y. Cao, Y. Wang, J. Liang, S. Liao, Y. Zhang, P. Li, and G. Sartoretti, "HEADER: Hierarchical robot exploration via attention-based deep reinforcement learning with expert-guided reward," 2025. [Online]. Available: <https://arxiv.org/abs/2510.15679>
- [11] Z. Li, K. Zheng, Y. Yuan, J. Huang, X. Zhang, J. Wu, and H. Cheng, "Learning to explore efficiently: Heterogeneous topological graphs and lightweight global reasoning for robotic exploration," *IEEE Robot. Automat. Lett.*, vol. 10, no. 12, pp. 12 357–12 364, 2025.
- [12] L. Jingsong, C. Yuhong, M. Yixiao, Z. Hanqi, and S. Guillaume, "HDPlanner: Advancing autonomous deployments in unknown environments through hierarchical decision networks," *IEEE Robot. Automat. Lett.*, vol. 10, no. 1, pp. 256–263, 2025.
- [13] X. Diao, W. Chi, and J. Wang, "Graph neural network based method for robot path planning," *Biomimetic Intell. Robot.*, vol. 4, no. 1, 2024.
- [14] Y. Cao, T. Hou, Y. Wang, X. Yi, and G. Sartoretti, "ARIADNE: A reinforcement learning approach using attention-based deep networks for exploration," in *Proc. of the IEEE Intl. Conf. on Robot. and Autom. (ICRA)*, 2023, pp. 10 219–10 225.
- [15] Y. Cao, R. Zhao, Y. Wang, B. Xiang, and G. Sartoretti, "Deep reinforcement learning-based large-scale robot exploration," *IEEE Robot. Automat. Lett.*, vol. 9, no. 5, pp. 4631–4638, 2024.
- [16] M. Sai, A. Deepesh, D. Laya, H. Mahantesh, and N. Balasubramaniam, "Challenges and opportunities in deep reinforcement learning with graph neural networks: A comprehensive review of algorithms and applications," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 11, pp. 15 051–15 071, 2024.
- [17] Y. Brian, "A frontier-based approach for autonomous exploration," in *Proc. IEEE Int. Symp. Comput. Intell. Robot. Automat. CIRA'97. New Comput. Princ. Robot. Automat.*, 1997, pp. 146–151.
- [18] J. Huang, B. Zhou, Z. Fan, Y. Zhu, Y. Jie, L. Li, and H. Cheng, "FAEL: Fast autonomous exploration for large-scale environments with a mobile robot," *IEEE Robot. Automat. Lett.*, vol. 8, no. 3, pp. 1667–1674, 2023.
- [19] Y. Zhang, X. Chen, C. Feng, B. Zhou, and S. Shen, "FALCON: Fast autonomous aerial exploration using coverage path guidance," *IEEE Trans. Robot.*, vol. 41, pp. 1365 – 1385, 2024.
- [20] S. Geng, Z. Ning, F. Zhang, and B. Zhou, "EPIC: A lightweight lidar-based aav exploration framework for large-scale scenarios," *IEEE Robot. Automat. Lett.*, vol. 10, no. 5, pp. 5090–5097, 2025.
- [21] S. Long, Y. Li, C. Wu, B. Xu, and W. Fan, "HPHS: Hierarchical planning based on hybrid frontier sampling for unknown environments exploration," in *Proc. of the IEEE/RSJ Intl. Conf. on Intell. Robots and Syst. (IROS)*, 2025, pp. 12 056–12 063.
- [22] X. Yang, Z. Ronghao, Z. Senlin, L. Meiqin, and H. Shoudong, "CARE: Confidence-rich autonomous robot exploration using bayesian kernel inference and optimization," *IEEE Robot. Automat. Lett.*, vol. 8, no. 10, pp. 6755–6762, 2023.
- [23] Z. Li, J. Xin, and N. Li, "Autonomous exploration and mapping for mobile robots via cumulative curriculum reinforcement learning," in *Proc. of the IEEE/RSJ Intl. Conf. on Intell. Robots and Syst. (IROS)*, 2023, pp. 7495–7500.
- [24] E. P. H. Alarcón, D. B. Ghavifekr, G. Baris, M. Mugnai, M. Satler, and C. A. Avizzano, "An efficient object-oriented exploration algorithm for unmanned aerial vehicles," in *Proc. of the Intl. Conf. on Unmanned Aircraft Syst. (ICUAS)*, 2021, pp. 330–337.
- [25] A. Dionigi, A. Devo, L. Guiducci, and G. Costante, "E-VAT: An asymmetric end-to-end approach to visual active exploration and tracking," *IEEE Robot. Automat. Lett.*, vol. 7, no. 2, pp. 4259–4266, 2022.
- [26] Y. Hu, S. Wang, Y. Xie, S. Zheng, P. Shi, I. Rudas, and X. Cheng, "Deep reinforcement learning-based mapless navigation for mobile robot in unknown environment with local optima," *IEEE Robot. Automat. Lett.*, vol. 10, no. 1, pp. 628–635, 2025.
- [27] Y. Wang, H. He, J. Liang, Y. Cao, R. Chakraborty, and G. Sartoretti, "CogniPlan: Uncertainty-guided path planning with conditional generative layout prediction," 2025. [Online]. Available: <https://arxiv.org/abs/2508.03027>
- [28] T. Noël, A. Lehuger, E. Marchand, and F. Chaumette, "Skeleton disk-graph roadmap: A sparse deterministic roadmap for safe 2d navigation and exploration," *IEEE Robot. Automat. Lett.*, vol. 9, no. 1, pp. 555–562, 2024.
- [29] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *Proc. Intl. Conf. Learn. Represent. (ICLR)*, 2017.
- [30] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 30, no. 4, pp. 1097–1109, 2018.
- [31] Z. Wang, S. Pan, K. Yu, W. Gao, and Z. Chen, "Data-driven modeling with deep koopman operator for robust path tracking of autonomous vehicles," *IEEE Trans. Ind. Electron.*, 2026, early Access.
- [32] P. Christodoulou, "Soft actor-critic for discrete action settings," 2019. [Online]. Available: <https://arxiv.org/abs/1910.07207>
- [33] W. Xu, Y. Cai, D. He, J. Lin, and F. Zhang, "FAST-LIO2: Fast direct lidar-inertial odometry," *IEEE Trans. Robot.*, vol. 38, no. 4, pp. 2053–2073, 2022.